

TestSplit Testing Document

Authors: Samson Oloruntola (22714745) & Marjia Siddik (22306501)

Supervisor: Dr Paul Clarke

Module: CSC1097 - Final Year Project

Submission Date: 01/05/2026

Table of Contents

1. Introduction.....	2
2. Testing Strategy.....	3
2.1 Test Layers.....	3
2.2 CI/CD Pipeline.....	4
2.2.1 Pipeline Configuration.....	4
2.2.2 Environment Configuration.....	5
2.3 Coverage Thresholds.....	5
3. Static Analysis.....	6
3.1 Linting.....	6
3.2 TypeScript Type Checking.....	6
3.3 Dependency Security Audit.....	6
3.4 Summary.....	7
3.5 Code Review.....	7
4. Unit Tests.....	8
4.1 Testing Tools and Libraries.....	8
4.2 What We Tested.....	9
4.3 Example Test Cases.....	10
4.3.1 LPT Scheduler: Dominant Outlier Test.....	10
4.3.2 JUnit XML Parser: Fully-Qualified Classname Test.....	11
4.4 Coverage Results.....	11
4.5 Frontend Unit Tests.....	12
5. Integration Tests.....	13
5.1 Testing Tools and Libraries.....	13
5.2 What We Tested.....	14
5.3 Example Test Case.....	14
5.4 Results.....	17
5.5 Challenges and Solutions.....	17
6. End-to-End Tests.....	18
6.1 Testing Tools and Libraries.....	18
6.2 What We Tested.....	19
6.3 Example Test Case.....	20
6.4 Results.....	21
6.5 Challenges and Solutions.....	22
7. Ad-Hoc Testing.....	23
7.1 Real-World Repository Testing.....	23
7.2 Merge Request Testing.....	24
7.3 Dashboard Manual Testing.....	25
8. References.....	25

1. Introduction

This document outlines the testing approach, processes, and results for TestSplit, a command-line tool and web dashboard that optimises CI pipeline performance by intelligently distributing test suites across parallel jobs using the Longest Processing Time (LPT) and MULTIFIT scheduling algorithms. TestSplit parses JUnit XML output from Maven projects, profiles historical test durations, and generates ready-to-use CI configuration files for GitHub Actions and GitLab CI.

Given that TestSplit is itself a tool designed to improve automated testing workflows, ensuring its own correctness and reliability was a core priority throughout development. The testing completed falls into two main categories:

1. **Automated Tests:** Unit tests, integration tests, and end-to-end browser tests, all executed automatically on every pipeline run via GitLab CI.
2. **Manual Tests:** Ad-hoc testing and real-world verification performed by running TestSplit against open-source Maven repositories to validate end-to-end behaviour in realistic conditions.

The automated test suite consists of 705 unit tests, 10 integration tests, and 12 Playwright end-to-end tests, all executed through the GitLab CI pipeline. The pipeline continues to cover build, unit testing, integration testing, and end-to-end browser testing stages.

2. Testing Strategy

The testing strategy for TestSplit was guided by the testing pyramid methodology, prioritising a large base of fast, isolated unit tests, supported by a smaller set of integration tests covering multi-component flows, and capped with end-to-end browser tests for the web dashboard. This structure reflects the practical trade-offs of each layer: unit tests are cheap to run and easy to debug, integration tests catch issues that unit tests cannot, and end-to-end tests validate the full user experience at the cost of speed and setup complexity.

Tests were written upfront alongside feature development, a test-driven approach that gave us confidence in each module before it was integrated into the broader system. This was especially important for the algorithm layer, where the correctness of the LPT and MULTIFIT schedulers needed to be verified independently before they were wired into the profiling and generation pipeline.

2.1 Test Layers

The test suite is organised into three layers:

- **Unit Tests:** 705 unit tests covering backend and frontend modules in isolation. Backend tests cover the algorithm, parser, profiler, generator, detector, runner, storage, CLI, and API modules using Jest. Frontend tests cover all four dashboard pages, shared components, custom hooks, and utility functions using Vitest with React Testing Library. Together, these are the largest and fastest layers, with the backend suite completing in under 10 seconds and the frontend suite in under 2 seconds.

- **Integration Tests:** 10 tests across 4 suites covering multi-step backend flows, CLI wiring, and end-to-end pipeline execution from XML parsing through to CI configuration generation. These include GitHub Actions and GitLab CI configuration generation checks using real fixture files and real file I/O, not mocked objects.
- **End-to-End Tests:** 12 Playwright browser tests covering the web dashboard UI, validating navigation, page rendering, data visualisation, theme persistence, scheduling-page behaviour, durations-page stability, and API error handling against a live seeded dashboard instance.

2.2 CI/CD Pipeline

All automated tests are executed in a GitLab CI pipeline triggered on every push. The pipeline is structured into sequential stages, ensuring that fast, cheap checks (linting, type checking) fail early before slower tests are run. End-to-end tests are gated to merge request events and pushes to the default branch only, preventing unnecessary overhead on every feature branch commit.

2.2.1 Pipeline Configuration

The pipeline is divided into four main stages, each building on the success of the previous:

Stage	Jobs	Purpose
Build	lint, typecheck_and_build	Runs ESLint, npm audit, TypeScript type checking, and a full build. Both jobs run in parallel.
Unit Tests	backend_tests, frontend_tests	backend_tests executes the full Jest backend suite with coverage reporting and JUnit XML artifact output. frontend_tests runs the Vitest frontend suite with lcov coverage. Both jobs run in parallel.
Integration Tests	integration_tests	Runs the Jest integration suite against real fixture files and CLI flows. JUnit XML is published as a pipeline artifact.
E2E Tests	e2e_tests	Runs the Playwright dashboard suite. Gated to merge requests and the default branch only to avoid overhead on every feature commit.

Table 1: Overview of CI/CD Pipeline Stages, Jobs, and Their Purpose

Each stage shares the following pipeline behaviours:

- **Dependency caching:** All stages cache node_modules/ keyed on package-lock.json to avoid redundant npm ci calls across pipeline runs.
- **Artifact retention:** Coverage reports, JUnit XML, and Playwright HTML reports are retained as pipeline artifacts for 30 days.

- **Coverage extraction:** A regex pattern in the backend_tests job extracts the overall backend coverage percentage from Jest output and displays it as a pipeline badge in GitLab. The frontend_tests job produces an lcov report under src/frontend/coverage/ retained as a 30-day artifact.
- **JUnit reporting:** Both unit and integration jobs publish JUnit XML to coverage/junit.xml, enabling GitLab to display per-test pass/fail results directly within merge request views.

2.2.2 Environment Configuration

Each pipeline stage runs inside a dedicated Docker image chosen to match the requirements of that stage:

Stage	Image	Rationale
Build, Unit, Integration	node:20.19.0-bullseye-slim	Slim Node.js 20 LTS image. Provides a reproducible, minimal environment for TypeScript compilation and Jest execution.
E2E Tests	mcr.microsoft.com/playwright:v1.59.0-jammy	Official Microsoft Playwright image with Chromium pre-installed. Eliminates browser installation overhead in the pipeline.

Table 2: CI/CD Pipeline Environment Configuration: Docker Images and Rationale

For local development, the full test suite can also be run inside Docker using the `npm run docker:test` command, providing an isolated environment that closely mirrors the CI pipeline:

```
npm run docker:test      # Runs the full test suite inside Docker
npm run cli -- <command> # Local CLI entrypoint for manual
verification
```

The E2E stage installs dependencies for both the root project and the frontend separately, then invokes Playwright with `CI=true` to suppress interactive output.

2.3 Coverage Thresholds

Coverage thresholds are enforced at the CI level via `jest.base.config.js`. The build fails if any threshold is not met, ensuring coverage cannot regress silently:

- 80% Lines
- 80% Functions

- 80% Branches
- 80% Statements

In practice, the current suite achieves 100% across all four metrics on every file, significantly exceeding these minimum thresholds.

3. Static Analysis

Static analysis was applied throughout development as a lightweight but high-value quality gate. Unlike runtime tests, static analysis catches issues at the source level without executing any code, making it cheap to run, fast to fail, and easy to enforce automatically in the CI pipeline. For TestSplit, three static analysis tools were configured and run on every push during the build stage.

3.1 Linting

ESLint is configured to run across the entire TypeScript codebase via the lint job in the build stage. It enforces a consistent code style, catches common programming mistakes, and flags patterns likely to cause bugs, such as unused variables, implicit any types, and unsafe equality comparisons.

Linting runs as a dedicated parallel job alongside `typecheck_and_build`, so it does not affect the overall pipeline duration. The lint job is configured to fail the pipeline on any ESLint error, enforcing that no linting violations can be merged into the codebase:

```
npm run lint
```

3.2 TypeScript Type Checking

TypeScript's static type checker is run explicitly as a separate step before the build, via `npm run typecheck`. This performs a full type-checking pass across the codebase without emitting any output files, catching type errors that would otherwise only surface at runtime.

For a project like TestSplit, which processes external data formats such as JUnit XML and Maven POM files, and generates structured YAML output, strong typing is particularly valuable. Type errors at module boundaries, such as a parser returning an unexpected shape or a scheduler receiving an incorrectly typed task list, are caught before tests even run:

```
npm run typecheck # Type-check only, no emit  
npm run build    # Full TypeScript compilation
```

3.3 Dependency Security Audit

`npm audit` is run as part of the lint job with the `--audit-level=high` flag. This checks all project dependencies against the npm advisory database and fails the pipeline if any dependency has a known vulnerability rated high or critical:

```
npm audit --audit-level=high
```

The high threshold was chosen deliberately; low and moderate-severity advisories are flagged in the output for awareness but do not block the build, striking a balance between security diligence and avoiding unnecessary pipeline noise from low-risk advisories in transitive dependencies.

3.4 Summary

Tool	Command	Pipeline Job	What It Catches
ESLint	npm run lint	lint	Style violations, unsafe patterns, unused variables
TypeScript	npm run typecheck	typecheck_and_build	Type errors, incorrect module interfaces, and missing types
npm audit	npm audit --audit-level=high	lint	High and critical severity dependency vulnerabilities

Table 3: Summary of CI/CD Pipeline Tools, Commands, and Their Validation Roles

3.5 Code Review

Code review was a mandatory part of the development workflow. All changes were introduced via GitLab merge requests, and no merge could be completed without explicit approval from the other team member. This applied to all features, bug fixes, and refactors; no direct pushes to the main branch were permitted.

The review process served as a complementary quality gate to the automated pipeline. Reviewers checked for correctness of logic, test coverage of new code, adherence to the module interfaces defined in the technical specification, and any concerns that static analysis tools would not catch, such as incorrect algorithmic assumptions or misleading variable naming.

A merge request could only be approved and merged once:

- All pipeline stages passed, including lint, type checking, unit tests, and integration tests.
- The other team member had reviewed the diff and left any comments they wanted addressed.
- The MR author had resolved all review comments.
- Explicit approval had been given by the reviewer in GitLab.

This process ensured that every piece of code in the repository had been seen by both developers before merging, reducing the risk of introducing bugs or architectural inconsistencies that automated tooling alone would not detect.

4. Unit Tests

The unit test suite is the largest and most comprehensive layer of TestSplit's testing strategy. It is split into two suites: a Jest-based backend suite covering all server-side modules, and a Vitest-based frontend suite covering the web dashboard. Each module is tested in isolation, with external dependencies mocked where necessary, ensuring that failures are localised and easy to diagnose. Tests were written upfront alongside development, giving confidence in each module's correctness before it was wired into the broader system.

4.1 Testing Tools and Libraries

The unit suite is built on the following tools:

- **Jest:** The core test runner and assertion library. Provides test organisation via describe/it blocks, built-in mocking with `jest.fn()` and `jest.spyOn()`, and coverage collection via the V8 provider.
- **ts-jest:** A Jest transformer that compiles TypeScript on the fly, allowing test files to be written in TypeScript without a separate compilation step. This ensures type safety is preserved in the test code itself.
- **jest.base.config.js:** A shared base configuration that enforces coverage collection from `src/backend/**/*.ts`, outputs JUnit XML to `coverage/junit.xml`, and enforces minimum coverage thresholds across all runs.
- **Vitest:** The test runner for the frontend suite, configured via `vite.config.ts`. Runs tests in a jsdom environment with globals enabled, providing a browser-like DOM without requiring a real browser. Used in place of Jest for the frontend because it integrates directly with Vite's module resolution and transformation pipeline.
- **React Testing Library:** Used alongside Vitest to render React components and hooks in jsdom. `render` is used for component tests, `renderHook` is used for custom hooks (`useApi`, `useCalibration`). Interactions are driven through accessible queries (`getByRole`, `getByText`) rather than implementation details.
- **@vitest/coverage-v8:** The coverage provider for the frontend suite, producing lcov reports scoped to `src/lib/`, `src/hooks/`, `src/components/`, and `src/pages/`.

Tests are run using the following commands:

```
npm test           # Run unit suite (alias for test:unit)
npm run test:unit  # Run unit suite with coverage
npm run test:all   # Run unit and integration suites together
```

4.2 What We Tested

The unit suite contains 41 test files grouped by module area. The table below summarises the coverage of each area, what was verified, and why it matters:

Module Area	Test Files	Verified Functionality
Algorithm	LPTScheduler, MULTIFITScheduler, SchedulingMetrics, SchedulerValidator, scheduler.property	Correctness of LPT and MULTIFIT scheduling, metrics computation (criticalPath, balanceRatio), edge cases including single tasks, dominant outliers, and equal-duration tasks.
Parser	JUnitXMLParser	JUnit XML parsing including nested suites, fallback naming, missing time attributes, fully-qualified Maven classnames, and lifecycle metadata extraction.
Profiler	Profiler, HistoricalProfiler, ProfilerValidator, MetadataValidator, EnvironmentCollector	Exponential smoothing across runs, instability and regression detection, environment metadata capture, and input validation.
Detector	JUnitDependencyDetector, TestNGDependencyDetector, PomParser, SuiteXMLParser, LifecycleDetector	Maven POM parsing, TestNG and JUnit dependency detection, lifecycle service extraction, and test ordering constraint detection.
Generator	CIGithubGenerator, CIGitlabGenerator, DockerfileGenerator, JobBuilder, JobGroupValidator, CISchemaValidator, ValidationFeatures, LifecycleStepGenerator, ProjectInspection	GitHub Actions and GitLab CI YAML generation, Dockerfile generation, YAML syntax and schema validation, CI config validation features, job group structure and artifact wiring.
Runner	ParallelRunner, WorkQueue, TimingFeedback, CoreAffinity	Parallel test execution, work queue behaviour, timing feedback collection, and CPU core affinity assignment.

Storage	FileStore, StoragePaths	Compressed JSON persistence, storage path resolution, and schema version compatibility.
API	server, server.startup	Fastify server startup, route registration, and endpoint response correctness for the dashboard API.
CLI	cli, validate, CIConfigReader	Command parsing, flag validation, and CI configuration reading.
Core	DetectionOrchestration, TestSplitEngine	Orchestration of the full detection pipeline across parser, profiler, detector, and scheduler modules, and correctness of the top-level engine flow.
Helpers & Utils	CommitTracker RunId, Terminal, cliColors,	Run ID generation, commit tracking, terminal output formatting, and statistical utility functions.

Table 4: Overview of Unit Test Coverage and Verified Functionality by Module

4.3 Example Test Cases

Two representative test cases are shown below, drawn from the algorithm and parser modules. These were chosen because they illustrate two distinct testing concerns central to TestSplit: algorithmic correctness under adversarial input, and accurate parsing of real-world Maven output.

4.3.1 LPT Scheduler: Dominant Outlier Test

A key correctness concern for the LPT scheduler is how it handles a task whose duration far exceeds those of all others, a common real-world scenario in which one test takes several minutes while the rest are sub-second. The test below verifies that the scheduler does not crash, correctly identifies the outlier as the critical path, and produces a balance ratio greater than 1 (indicating the outlier dominates one job while the others remain underloaded):

```
it('handles a dominant outlier task without crashing', () => {
  const tasks: Task[] = [
    { id: 'A', duration: 100 },
    { id: 'B', duration: 1 },
    { id: 'C', duration: 1 },
    { id: 'D', duration: 1 },
  ];
  const result = new LPTScheduler().schedule(tasks, 2);
```

```
expect(result.metrics.criticalPath).toBe(100);
expect(result.metrics.balanceRatio).toBeGreaterThan(1);
});
```

Listing 1: LPT Scheduler (Dominant Outlier) Test Case

This test is significant because a dominant outlier is the worst-case input for any scheduling algorithm; no algorithm can distribute a single 100-unit task across two jobs. The test verifies that the scheduler behaves correctly and predictably in this scenario rather than producing an incorrect distribution or throwing an error.

4.3.2 JUnit XML Parser: Fully-Qualified Classname Test

Maven's Surefire plugin produces JUnit XML reports in which test names are stored as fully qualified Java class names concatenated with the method name. The parser must correctly separate these into their constituent parts, the class name and the method name, so that the profiler can group tests accurately. The test below verifies this behaviour against a real Maven fixture file:

```
test('preserves fully-qualified Maven classname separately from method name', () => {
  const results = parseJUnitXML(fixture('maven-fqn.xml'));

  expect(results).toHaveLength(2);
  expect(results[0].name).toBe(
    'org.apache.commons.lang3.StringUtilsTest.testIsEmpty',
  );
  expect(results[0].classname).toBe(
    'org.apache.commons.lang3.StringUtilsTest',
  );
  expect(results[1].classname).toBe(
    'org.apache.commons.lang3.StringUtilsTest',
  );
});
```

Listing 2: JUnit XML Parser (Fully-Qualified Classname) Test Case

This test uses a real Maven Surefire XML fixture modelled on output from apache/commons-lang3, one of the open-source repositories used for manual validation. Correct classname preservation is critical because the profiler uses classnames to group historical timings; incorrect parsing here would cause incorrect scheduling decisions downstream.

4.4 Coverage Results

The unit suite achieves 100% coverage across all four metrics on every file in the codebase. This was not a target set at the outset; it emerged from the test-driven approach: writing tests for each module before integration naturally drove coverage to its ceiling.

The full results from the latest run are summarised below:

Metric	Statements	Branches	Functions	Lines
Coverage	100%	100%	100%	100%
Enforced threshold	80%	80%	80%	80%

Table 5: Test Coverage Results and Enforced Thresholds

The suite now achieves 100% coverage across the codebase for statements, branches, functions, and lines. This confirms that the current automated test suite covers all measured execution paths.

4.5 Frontend Unit Tests

The frontend suite contains 16 test files and 101 tests, all located under `src/frontend/__tests__` and organised to mirror the source tree. The table below summarises coverage by area:

Area	Test Files	Verified Functionality
Pages	Overview, Durations, Scheduling, Instability	Each page renders without error, displays the correct section headings, and shows an appropriate empty state when no data is returned from the API.
Components	Sidebar, PageHeader, PageErrorState, StatCard, StatusBadge, TestRow, InstabilityRow, ExportButton, PageLoadingSkeleton	Correct rendering given prop combinations, active state highlighting, error and loading states, and conditional style application.
Hooks	useApi, useCalibration, useTheme	SWR-based data fetching with loading and error states, cache isolation between tests using a fresh SWRConfig provider and new Map() per test, and calibration state transitions.
Libs	utils, testStatus, testName	percentageChange, detectRegression, formatRunLabel, downloadJson, colour utilities for test status and dim variants, and method/class name extraction.

Table 6: Overview of Frontend Unit Test Coverage and Verified Functionality by Area

Recharts and motion/react are mocked at the module level to avoid SVG rendering issues in jsdom. SWR's global cache is cleared between tests using a SWRConfig wrapper that provides a fresh Map, preventing state leakage across test cases.

4.6 Challenges and Solutions

- **Mocking file system and process I/O:** Several modules, such as FileStore and ParallelRunner, interact directly with the file system and child processes. Jest's module mocking system was used to replace these with controlled in-memory implementations, keeping unit tests fast and deterministic without requiring real disk access.
- **Testing concurrent behaviour in ParallelRunner:** The parallel runner uses worker threads to execute tests concurrently. Unit testing concurrent code is inherently difficult. The WorkQueue was tested independently with deterministic inputs, and the ParallelRunner was tested using controlled fixtures that simulate worker completion in a predictable order.
- **Property-based testing for the scheduler:** The scheduler.property.test.ts file uses property-based testing to verify that LPT always produces a valid distribution regardless of input shape, including empty task lists, single tasks, and lists where all tasks have equal duration. This caught several edge cases that example-based tests alone would not have covered.

While unit tests verify individual modules in isolation, integration tests verify that those modules work correctly together under real conditions. TestSplit's integration suite focuses on multi-step backend flows, CLI wiring, and full pipeline execution using real fixture files, real file I/O, and real process invocations rather than mocked objects. This distinction is important: an integration test failure indicates a problem at a module boundary, not within a single module.

5. Integration Tests

5.1 Testing Tools and Libraries

The integration suite uses the following tools:

- **Jest:** The same test runner used for unit tests, configured separately via `jest.integration.config.js`. Coverage enforcement is disabled for integration tests, as the suite focuses on the correctness of behaviour rather than code coverage metrics.
- **ts-node:** Used to invoke the CLI directly from integration tests via Node's `spawnSync`, transpiling TypeScript on the fly without a separate build step. This allows the tests to exercise the real CLI entry point as a user would.
- **spawnSync:** Node's synchronous child process API, used to invoke CLI commands and capture stdout, stderr, and exit codes for assertion.

- **yaml:** Used to parse the generated YAML output files and assert their structure, job names, step contents, artifact wiring, and dependency declarations.

Integration tests are run using:

```
npm run test:integration
```

5.2 What We Tested

The integration suite contains 4 test suites, totalling 10 tests. The table below summarises each suite:

Test Suite	Tests	Verified Functionality
generate-config.e2e.test.ts	CLI wiring and GitHub Actions YAML generation from real JUnit XML fixtures and existing workflow files	End-to-end CLI execution, output file creation, YAML structure, job naming, build/test job wiring, artifact upload/download, needs dependencies, and -Dtest= subset injection. Verifies both GitHub Actions and GitLab CI generation from real fixture files, including the new gitlab-ci.yml fixture.
generate-dockerfile.e2e.test.ts	CLI Dockerfile generation from different POM shapes	Java version extraction from compiler plugin, java.version property, default fallback to Java 21, and correct use of mvn vs ./mvnw depending on wrapper presence.
TestSplitEngine.test.ts	Core engine integration across parser, profiler, scheduler, and persistence	Profile and distribution output are written to the configured data directory, and the correct engine behaviour is observed when processing real XML fixture files.
EndToEnd.test.ts	Broader pipeline coverage across all layers	Full parse → profile → schedule → CI generation flow executed in sequence, verifying that output from each stage is correctly consumed by the next.

Table 7: Overview of Integration Test Suites and Verified Functionality

5.3 Example Test Case

The test below is drawn from generate-config.e2e.test.ts and represents the most comprehensive integration test in the suite. It invokes the CLI directly via spawnSync using real JUnit XML output from

apache/commons-lang as input, and then parses and asserts the structure of the generated GitHub Actions YAML file in detail:

```

test('handles commons-lang style Maven CI (no "test" keyword in Maven step)', () => {
  const tempDir = fs.mkdtempSync(path.join(os.tmpdir(), 'testsplit-generate-'));
  const outPath = path.join(tempDir, 'testsplit.yml');

  try {
    const result = spawnSync(
      TS_NODE,
      [
        '--transpile-only', CLI_PATH, 'generate-config',
        '--junit', COMMONS_LANG_JUNIT,
        '--jobs', '3',
        '--platform', 'github',
        '--from', COMMONS_LANG_CI,
        '--out', outPath,
      ],
      SPAWN_OPTS,
    );

    expect(result.status).toBe(0);
    expect(fs.existsSync(outPath)).toBe(true);

    const parsed = YAML.parse(fs.readFileSync(outPath, 'utf-8'));
    expect(parsed.jobs).toBeDefined();

    const jobNames = Object.keys(parsed.jobs);
    expect(jobNames).toHaveLength(4); // build + 3 test jobs
    expect(jobNames).toContain('build');

    const testJobNames = jobNames.filter((n) => n.startsWith('test-job-'));
    expect(testJobNames).toHaveLength(3);

    // Build job must compile with -DskipTests and upload artifacts
    const buildSteps = parsed.jobs['build'].steps as any[];
    const buildMavenStep = buildSteps.find(
      (s: any) => typeof s.run === 'string' && s.run.includes('mvn')
    );
    expect(buildMavenStep.run).toContain('-DskipTests');
    expect(buildSteps.some(
      (s: any) => s.uses?.startsWith('actions/upload-artifact')
    )).toBe(true);

    for (const jobName of testJobNames) {

```

```

const steps = parsed.jobs[jobName].steps as any[];
expect(steps.some(
  (s: any) => s.uses?.startsWith('actions/checkout')
)).toBe(true);
expect(steps.some(
  (s: any) => s.uses?.startsWith('actions/download-artifact')
)).toBe(true);
const testStep = steps.find(
  (s: any) => typeof s.run === 'string' && s.run.includes('-Dtest=')
);
expect(testStep.run).toMatch(/mvn test -Dtest=.\+ -DfailIfNoTests=false/);
expect(parsed.jobs[jobName].needs).toContain('build');
}
} finally {
  fs.rmSync(tempDir, { recursive: true, force: true });
}
});

```

Listing 3: Example Test Case from generate-config.e2e.test.ts

This test exercises the full CLI pipeline in a single assertion chain. It verifies not just that the output file is created, but that its contents are structurally correct, the right number of jobs, correct Maven flags, proper artifact upload and download wiring between the build and test jobs, correct needs declarations, and the `-Dtest=` subset pattern used to split tests across parallel jobs. It uses real JUnit XML output from `apache/commons-lang`, the same open-source repository used as a benchmark throughout the project.

5.4 Results

The latest integration test run produced the following results:

```

Test Suites: 4 passed, 4 total
Tests:      10 passed, 10 total
Time:       162.231 s

```

All 4 suites and 10 tests passed. The total runtime of 162 seconds is discussed in the challenges section below.

5.5 Challenges and Solutions

- **Integration test runtime:** The `generate-config.e2e.test.ts` suite accounts for the majority of the 162-second total runtime. This is because it invokes the CLI via `spawnSync` with `ts-node`, which incurs TypeScript compilation overhead on each invocation, and processes real JUnit XML from `apache/commons-lang` rather than minimal fixtures. This runtime is accepted as a trade-off for

testing the CLI against realistic input. The integration suite is kept as a separate pipeline stage from unit tests precisely to avoid slowing down the faster unit feedback loop.

- **Schema validation warning in fixtures:** During the TestSplitEngine and EndToEnd integration tests, the JUnit XML parser emits a console warning for the basic.xml fixture file, which is missing the required tests attribute on its <testsuite> element. This warning is intentional; the parser is designed to handle malformed XML gracefully rather than throw an error. The test still passes, and the warning confirms that the schema validation logic is working correctly by flagging the issue without crashing.
- **Temporary directory management:** CLI integration tests write output files to temporary directories created with fs.mkdtempSync. A try/finally block ensures these directories are always cleaned up with fs.rmSync after each test, regardless of whether the test passes or fails. This prevents stale test artifacts from accumulating on the file system across repeated test runs.

6. End-to-End Tests

The end-to-end test suite validates the TestSplit web dashboard from the perspective of a real user interacting with a live browser. Where unit and integration tests verify backend logic, the E2E suite verifies that the frontend correctly renders profiling data, responds to navigation, and presents meaningful visualisations, concerns that no amount of backend testing can address alone.

6.1 Testing Tools and Libraries

The E2E suite is built on the following tools:

Playwright: Microsoft's browser automation framework, used to drive a real Chromium instance against the live dashboard. Playwright was chosen over DOM emulation libraries such as Happy-dom or Jsdom because the dashboard relies on real browser rendering for its data visualisations, which do not function correctly in simulated DOM environments.

Chromium: The only browser target configured in playwright.config.ts. The official Microsoft Playwright Docker image (mcr.microsoft.com/playwright:v1.59.0-jammy) ships with Chromium pre-installed, eliminating the need to install a browser in the CI pipeline.

Playwright web server: The Playwright configuration launches the dashboard automatically before the test suite runs, serving it on http://localhost:5173 via the existing dashboard command. Tests do not need to manage the server lifecycle manually.

Playwright HTML reporter: Test results and failure screenshots are written to coverage/playwright-report and retained as a 30-day pipeline artifact, available for review directly in GitLab.

The E2E suite is run using:

```
npm run test:e2e      # Runs Playwright suite (installs Chromium first via pretest:e2e)
npm run test:e2e:ui   # Runs in interactive UI mode for local debugging
```

A pretest:e2e hook is configured in package.json to run npx playwright install chromium automatically before the suite executes, ensuring the correct browser binary is always available without requiring a manual installation step.

6.2 What We Tested

All twelve E2E tests are contained in a single file, dashboard.spec.ts, and are grouped under a Dashboard navigation describe block. The suite seeds real profiling data in a beforeAll hook by running the profile command via ts-node before any browser assertions are made.

Test	Verified Functionality
Loads the Overview page by default	The dashboard renders the Overview page on initial load without error.
Navigates to all four pages without crashing	Navigation across the Overview, Durations, Scheduling, and Instability pages succeeds without a runtime error or a blank render.
Sidebar highlights the active page.	The active sidebar item receives the correct highlighted state when each page is visited.
The instability scatter plot renders with data points	The Instability page renders an SVG scatter plot containing actual data points, confirming that the visualisation layer correctly consumes the profiled data from the API.
Loads with real profiled data after a profile run	The dashboard loads real profiling data from the API, displays a Duration Trend, shows no empty-state messages, and renders a run badge with a count greater than zero.
Theme toggle	Verifies that toggling the theme applies the light class and that it persists across reloads.
Scheduling page statistics	Confirms that scheduling stats render correctly.
Scheduling job count badge	Confirms that the job count badge is visible.

Scheduling export button	Confirms that the export button is visible.
Durations page render	Confirms that the Durations page renders without crashing.
API error state	Confirms that the dashboard shows an error message when the API is unreachable.
Expanded dashboard coverage	Broadens dashboard regression coverage across navigation, rendering, state, and API failure handling.

Table 8: Overview of Frontend End-to-End Tests and Verified User Interface Behaviour

6.3 Example Test Case

The test below is the most comprehensive in the suite. Rather than testing a single UI element, it verifies that the dashboard is correctly wired to a live data source, polling the `/api/summary` endpoint until data is available, then asserting that the UI reflects that data meaningfully:

```

test('loads with real profiled data after a profile run', async ({ page }) => {
  const summaryStatus = async () => {
    const response = await page.request.get('/api/summary');
    return response.status();
  };

  if ((await summaryStatus()) === 404) {
    seedDockerApiDataIfAvailable();
  }

  await expect.poll(summaryStatus, {
    timeout: 20000,
    intervals: [500, 1000, 2000],
  }).toBe(200);

  await page.goto('/');
  await expect(page.getByText(/Duration Trend/i)).toBeVisible({ timeout: 10000 });
  await expect(page.getByText(/No profiling data found/i)).toHaveCount(0);
  await expect(page.getByText(/No distribution data found/i)).toHaveCount(0);

  const runBadge = page.getByText(/^RUN\s+\d+$/);
  await expect(runBadge).toBeVisible();

  const runBadgeText = await runBadge.innerText();
  const runCount = Number.parseInt(runBadgeText.replace(/D/g, ''), 10);
  expect(runCount).toBeGreaterThan(0);
});

```

Listing 4: Example Test Case from dashboard.spec.ts

Several aspects of this test are worth highlighting. The `expect.poll` call retries the API check at increasing intervals up to a 20-second timeout, allowing time for the profile command to seed data before assertions begin. The test also explicitly asserts the absence of empty-state messages, verifying not just that the page loads, but that it is populated with real data. Finally, the run badge assertion confirms that the profiling run count is a positive integer, validating end-to-end data flow from the profiler through the API to the UI.

6.4 Results

The latest E2E run produced the following results:

Running 12 tests using 4 workers

- ✓ 1 [chromium] dashboard.spec.ts:99 > API error state > shows an error state when the API is unreachable (343ms)
- ✓ 2 [chromium] dashboard.spec.ts:46 > Dashboard navigation > sidebar highlights the active page (441ms)
- ✓ 3 [chromium] dashboard.spec.ts:87 > Theme toggle > toggles light mode class on the html element (535ms)
- ✓ 4 [chromium] dashboard.spec.ts:75 > Scheduling page > shows the correct job count badge (1.0s)
- ✓ 5 [chromium] dashboard.spec.ts:31 > Dashboard navigation > loads the Overview page by default (228ms)
- ✓ 6 [chromium] dashboard.spec.ts:54 > Dashboard navigation > Instability scatter plot renders with data points (1.2s)
- ✓ 7 [chromium] dashboard.spec.ts:93 > Theme toggle > persists theme preference across page reloads (402ms)
- ✓ 8 [chromium] dashboard.spec.ts:37 > Dashboard navigation > navigates to all four pages without crashing (403ms)
- ✓ 9 [chromium] dashboard.spec.ts:59 > Scheduling page > renders the Scheduling page stat cards (1.3s)
- ✓ 10 [chromium] dashboard.spec.ts:81 > Scheduling page > export button is visible on the Scheduling page (881ms)
- ✓ 11 [chromium] dashboard.spec.ts:71 > Dashboard navigation > loads with real profiled data after a profile run (852ms)
- ✓ 12 [chromium] dashboard.spec.ts:105 > Durations page > renders the Durations page without crashing (221ms)

12 passed (7.6s)

All 12 E2E tests passed on Chromium using 4 parallel Playwright workers in 7.6 seconds. The updated suite now covers dashboard navigation, real profiled data rendering, scheduling page assertions, theme toggle behaviour (including persistence), durations page stability, and API error state handling, while still keeping the full-suite runtime under 10 seconds.

6.5 Challenges and Solutions

- **DOM emulation incompatibility:** An initial investigation into using Vitest with a DOM emulation library such as Happy-dom or Jsdom was abandoned because the dashboard's data visualisation components do not render correctly outside a real browser environment. Playwright with a real Chromium instance was adopted instead, which resolved this entirely at the cost of slightly more complex test infrastructure.

- **Data seeding before assertions:** Browser tests that assert on live data require that data to exist before the test runs. The suite handles this by running the profile command in a beforeAll hook to seed the local data store, and then using expect.poll to wait for the /api/summary endpoint to return 200 before proceeding. This makes the tests resilient to variable startup times without introducing arbitrary sleep calls.
- **Browser installation in CI:** The official Playwright Docker image used in the CI pipeline ships with Chromium pre-installed, so no browser installation is needed there. For local runs, a pretest:e2e hook is configured to run npx playwright install chromium automatically before the suite executes, ensuring the correct browser binary is always present without requiring a manual step.

7. Ad-Hoc Testing

Ad-hoc testing complemented the automated suite throughout development. While the pipeline enforced correctness at the module and integration level, manual testing was used to verify real-world behaviour against genuine Maven projects, catching issues that fixture-based tests alone could not surface, and to validate new features before raising a merge request.

7.1 Real-World Repository Testing

TestSplit was run manually against well-known open-source Maven repositories throughout development. These repositories were chosen because they represent a range of project sizes and test suite structures, from small focused libraries to large multi-module projects with hundreds of tests, providing realistic and diverse inputs that synthetic fixtures cannot replicate.

Repository	Test Count	Verified Functionality
apache/commons-lang	64,725	The primary benchmark throughout development. Used to validate LPT scheduling, CI config generation, and Dockerfile generation against a large, realistic Maven project. Also used as the fixture source for generate-config integration tests.
jhy/jsoup	1,853	Validated correct handling of a smaller, focused test suite. Confirmed that TestSplit produces sensible distributions when the number of tests is modest relative to the requested number of parallel jobs.

bonigarcia/mastering-junit5	2	Validated correct handling of automated Selenium system tests. Confirmed that TestSplit's generated CI config preserves Xvfb browser setup steps and does not break browser-dependent tests. Also exposed and fixed a generator bug where project detection (Dockerfile, docker-compose.yml) incorrectly resolved against the working directory rather than the target project
------------------------------------	---	--

Table 9: Overview of External Repository Benchmarks and Validation Scope

For each repository, the typical manual testing workflow was as follows:

```
# 1. Clone the target repository and run its test suite to produce JUnit XML
mvn test

# 2. Profile the JUnit output with TestSplit
npm run cli -- profile --junit target/surefire-reports/

# 3. Generate a CI configuration
npm run cli -- generate-config --junit target/surefire-reports/ --jobs 4 --platform github

# 4. Inspect the generated YAML and verify job structure, artifact wiring,
# and -Dtest= subsets manually
```

This workflow was repeated across multiple runs for each repository to verify that historical profiling accumulated correctly over time and that the profiler's exponential smoothing produced stable, converging duration estimates rather than noisy values that changed significantly between runs.

7.2 Merge Request Testing

All code changes were introduced through GitLab merge requests. Before raising an MR, the author was required to manually test the new or changed functionality on their local machine, running the CLI against a real repository, inspecting the output, and verifying that no existing behaviour had regressed.

The reviewer also tested the changes on their own machine before approving. Because the two developers used different machines and operating systems, this dual-device testing provided an additional layer of confidence that the tool behaved consistently across environments and was not relying on any machine-specific configuration.

The manual pre-merge checklist followed by both developers was:

- **Run the full automated test suite:** Confirm that all unit and integration tests pass locally before pushing.
- **Test the CLI against a real repository:** Running the affected commands against at least one of the benchmark repositories and inspecting the output manually.
- **Inspect generated output:** Reading the generated YAML or Dockerfile to confirm it was structurally correct and contained the expected content.
- **Check for regressions:** Verifying that any previously working flows still produced the same output after the change.

Only after both the automated pipeline and manual verification had passed was a merge request approved and merged. This process ensured that no change entered the main branch without having been validated by both developers on independent machines.

7.3 Dashboard Manual Testing

The web dashboard was tested manually throughout development by seeding it with real profiling data from the benchmark repositories and visually inspecting the rendered output. This complemented the Playwright E2E suite by catching visual and layout regressions that automated assertions might not detect, such as charts rendering with incorrect scale, labels overlapping, or panels displaying stale data after a profiling update.

Manual dashboard testing was performed after every significant change to the frontend or API layer, using the local development server:

```
npm run cli -- dashboard      # Launch dashboard with live profiling data
```

This form of testing was particularly useful during UI improvements in response to user feedback, where visual changes needed to be assessed subjectively rather than solely through automated assertions.

8. References

- [1] Fowler, M. "*Test Pyramid*". <https://martinfowler.com/bliki/TestPyramid.html>. Accessed: April 02, 2026.
- [2] Jest Contributors. "*Jest — Delightful JavaScript Testing*". <https://jestjs.io>. Accessed: April 02, 2026.
- [3] Microsoft. "*Playwright — Fast and Reliable End-to-End Testing for Modern Web Apps*". <https://playwright.dev>. Accessed: April 02, 2026.
- [4] Kulshekhar, K.. "*ts-jest — TypeScript preprocessor with source map support for Jest*". <https://kulshekhar.github.io/ts-jest>. Accessed: April 02, 2026.
- [5] GitLab Inc.. "*GitLab CI/CD Documentation*". <https://docs.gitlab.com/ee/ci/>. Accessed: April 02, 2026.
- [6] Google Testing Blog. "*Just Say No to More End-to-End Tests*". <https://testing.googleblog.com/2015/04/just-say-no-to-more-end-to-end-tests.html>. Accessed: April 02, 2026.
- [7] The Apache Software Foundation. "*apache/commons-lang — Apache Commons Lang*". <https://github.com/apache/commons-lang>. Accessed: April 02, 2026.
- [8] Hedley, J.. "*jhy/jsoup — Java HTML Parser*". <https://github.com/jhy/jsoup>. Accessed: April 02, 2026.